React.memo, PureComponent & Re-render Prevention

5 min read Updated 16 Jul 2026 Topic 15 of 62

2.5 - React.memo, PureComponent & Re-render Prevention

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What is React.memo?"	🟡	★★★★☆
"What is PureComponent?"	🟡	★★★☆☆
"How do you prevent unnecessary re-renders?"	🟡🔴	★★★★★
"What is shallow comparison?"	🟡	★★★☆☆
"When does React.memo NOT work?"	🟡🔴	★★★★☆

Why Components Re-render

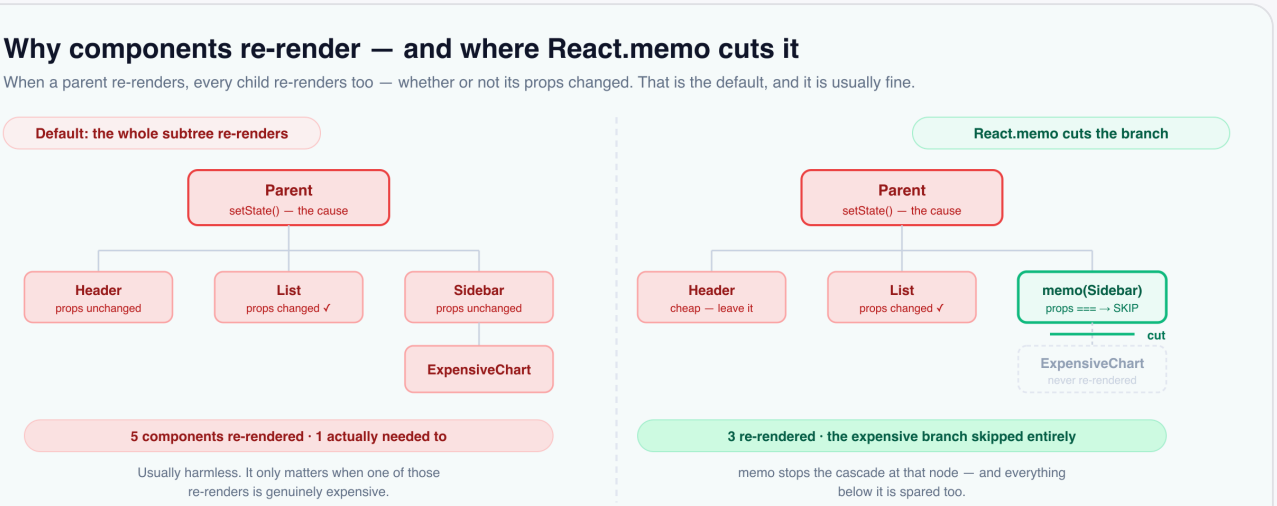
By default, when a parent component re-renders, **all of its children re-render too**, regardless of whether their props changed. This is React's default behavior and it's usually fine. But for expensive components, it's wasteful.

JSX

Copy

```jsx
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <ExpensiveChild name="Alice" />
      {/* ExpensiveChild re-renders on every click even though "Alice" nev
    </div>
  );
}
```



React.memo: Memoizing Functional Components

JSX Copy

```jsx
const ExpensiveChild = React.memo(function ExpensiveChild({ name }) {
  console.log('ExpensiveChild rendered');
  return <p>Hello, {name}</p>;
});

// Now ExpensiveChild only re-renders when `name` changes, not when Parent
```

How it decides: shallow comparison

React.memo compares each prop using `===` (strict equality). For primitives (strings, numbers, booleans), this works perfectly. For objects and arrays, it only checks if the reference is the same, not whether the contents are identical.

JSX Copy

```jsx
// ✅ Primitive props: shallow comparison works
<MemoChild name="Alice" count={5} />

// ❌ New object every render: defeats React.memo
<MemoChild style={{ color: 'red' }} />

// ❌ New array every render: defeats React.memo
<MemoChild items={[1, 2, 3]} />

// ❌ Inline function: new reference every render
<MemoChild onClick={() => doSomething()} />
```

To fix these, use `useMemo` for objects/arrays and `useCallback` for functions (covered in 2.3).

Custom comparison function

You can provide your own comparison function as the second argument to React.memo:

```jsx
const UserCard = React.memo(
  function UserCard({ user }) {
    return <div>{user.name} ({user.age})</div>;
  },
  (prevProps, nextProps) => {
    // Return true to SKIP re-render, false to allow it
    return prevProps.user.id === nextProps.user.id
        && prevProps.user.name === nextProps.user.name;
  }
);
```

Use this sparingly. If your custom comparison is wrong, you'll have stale UI bugs that are very hard to track down.

PureComponent: The Class Equivalent

`PureComponent` is the class component version of React.memo. It implements `shouldComponentUpdate` with a shallow props and state comparison automatically.

JSX Copy

```jsx
// Class: PureComponent
class Greeting extends React.PureComponent {
  render() {
    console.log('Greeting rendered');
    return <h3>Hello, {this.props.name}</h3>;
  }
}

// Functional: React.memo (equivalent)
const Greeting = React.memo(function Greeting({ name }) {
  console.log('Greeting rendered');
  return <h3>Hello, {name}</h3>;
});
```

In new code, use React.memo. PureComponent is for legacy class components you haven't migrated yet.

The Complete Re-render Prevention Toolkit

Tool	What it prevents	How to use
`React.memo`	Component re-render when props didn't change	Wrap the component
`useMemo`	Recalculation of expensive values	Wrap the computation
`useCallback`	Recreation of function references	Wrap the function
Move state down	Unrelated components re-rendering	Keep state close to whe
Split components	Large re-renders	Break big components i

The last two are free and don't add any code complexity. Try them first before reaching for memoization hooks.

Common Mistakes

Mistake 1: Using React.memo on every component. Most components are fast enough that the overhead of memoization (comparing all props) is more expensive than just re-

rendering. Profile first.

~~Mistake 2: Memoizing the child but passing new object/function references.~~ React.memo becomes useless if any prop changes on every render.

Mistake 3: Thinking React.memo makes the component "pure." It only prevents re-renders from parent changes. If the component's own state changes, it still re-renders (which is correct behavior).

🔥 Output Question

JSX Copy

```jsx
const Child = React.memo(({ user }) => {
  console.log('Child rendered');
  return <p>{user.name}</p>;
});

function Parent() {
  const [count, setCount] = useState(0);
  const user = { name: 'Alice' };

  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Count: {count}</button>
      <Child user={user} />
    </div>
  );
}
```

Does Child re-render when count changes?

Yes. Every render creates a new `user` object. Even though its contents are identical, React.memo's shallow comparison sees a different reference. Fix: move `user` outside the component or wrap it in `useMemo`.

> "React.memo is a higher-order component that memoizes a functional component, preventing re-renders when props haven't changed. It uses shallow comparison by default, which means it compares primitive values by value and objects by reference. This is why you need useMemo for objects and useCallback for functions passed as props. PureComponent is the class equivalent. The important thing is to not use these everywhere. Most components don't need memoization. I always profile with React DevTools first to identify actual bottlenecks, then apply memoization selectively."

✅ Key Takeaways

- **React.memo** prevents re-renders when props haven't changed (shallow comparison).
- **PureComponent** is the class equivalent. Use React.memo for new code.
- Shallow comparison: primitives by value, objects/arrays/functions by reference.
- New object or function on every render **defeats** React.memo. Use useMemo and useCallback to stabilize references.
- Don't memoize everything. Profile first, optimize only what's slow.
- Moving state down and splitting components are free optimizations that don't add complexity.

✨ QUICK CHECK — no marks, just for you.

1. A Parent component declares `const user = { name: 'Alice' };` inside its body and passes it to `<Child user={user} />`, where Child is wrapped in React.memo. When Parent re-renders because its own count state changed, does Child re-render, and why?

- ○ No — React.memo blocks all re-renders that originate from a parent, regardless of what props are passed

- Yes — each Parent render creates a brand-new user object, so React.memo's shallow comparison sees a different reference
- No — React.memo compares the object's contents, and name is still 'Alice', so the props are considered unchanged

Check answer

2. You pass a custom comparison function as the second argument to React.memo. What must that function return in order for React to SKIP re-rendering the component?

- undefined — returning nothing signals no change
- false
- true
- The previous props object

Check answer

3. A teammate says "I wrapped this component in React.memo, so now it won't re-render at all." According to the lesson, what is wrong with that claim?

- React.memo only prevents re-renders caused by parent changes; if the component's own state changes it still re-renders
- React.memo only works on class components, so wrapping a functional component has no effect at all
- React.memo prevents re-renders from the component's own state but not from parent re-renders

React.memo permanently freezes the component's output until it is unmounted and remounted

Check answer

Previous
Custom Hooks: Building Reusable Logic

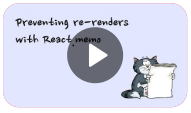
Next
useLayoutEffect vs useEffect

LEARN MORE

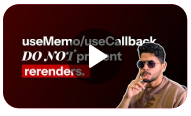
ARTICLES & DOCS

- memo — API reference
  react.dev
- PureComponent — API reference
  react.dev

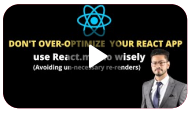
VIDEOS



- Preventing re-renders with React.memo
  YouTube



- useMemo & useCallback do NOT prevent re-renders
  YouTube



- Optimising with React.memo
  YouTube

Hand-picked references for this lesson.